DUY TAN UNIVERSITY

Da Nang

DS 423 – Machine Learning with Large Datasets

Emotion Detection in Tweets

Group 22 – NLP

Member	Student ID
Lâm Nhật Huy	29211465364
Nguyễn Thị Lam Giang	29201458314

June 2026

Contents

Abstract

Social-media platforms generate vast volumes of short text that reflect how people feel about products, events, and public issues. This project addresses the need to automatically detect emotions—such as joy, anger, sadness, fear, love, and surprise—from short social-media text. Such capability could support applications such as brand monitoring and public-opinion analysis; however, our implementation is an offline notebook-based classifier rather than a deployed real-time system. We built a multi-class text classifier using the *Emotion* dataset from Hugging Face, applying text cleaning, TF-IDF feature extraction, and Logistic Regression with hyperparameter tuning via stratified cross-validation. On the held-out test set of 2,000 tweets, the best model achieved **90.25% accuracy** and a **weighted F1-score of 0.9043**, substantially outperforming a majority-class baseline (34.75% accuracy). The strongest performance was observed for *sadness* and *joy*, while semantically similar or minority classes such as *love* and *surprise* were more challenging.

1 Introduction and Problem Statement

Brands, governments, and researchers increasingly monitor public emotion on social media to understand audience reactions, detect crises early, and measure campaign effectiveness. Unlike coarse sentiment analysis (positive/negative), **emotion detection** assigns fine-grained labels such as joy, anger, sadness, fear, love, and surprise to each post. In practice, such systems could be integrated into monitoring dashboards; our project focuses on building and evaluating the core classification pipeline in a Jupyter notebook.

Following the project requirement, this work builds a complete multi-class classifier that labels each input text with one emotion category and evaluates the model using accuracy, precision, recall, F1-score, and a confusion matrix.

The practical challenge is that tweets are short, informal, and noisy: they contain slang, misspellings, URLs, mentions, and hashtags. A reliable automated system must preprocess this text, learn discriminative patterns, and generalize to unseen posts.

Objective. Build a complete, runnable pipeline that:

- loads and explores the dair-ai Emotion dataset;
- preprocesses raw tweet text;
- trains a supervised multi-class classifier with hyperparameter tuning;
- evaluates performance using accuracy, precision, recall, F1-score, and a confusion matrix.

2 Dataset

2.1 Source and Splits

We used the **Emotion Dataset (dair-ai)** hosted on Hugging Face (`https://huggingface.co/datasets/dair-ai/emotion`). The dataset contains English tweets labeled with one of six emotions. It is split into training, validation, and test sets as shown in Table 1.

Table 1: Dataset splits

Split	Samples	Purpose
Train	16,000	Model training and cross-validation
Validation	2,000	Development / sanity check
Test	2,000	Final evaluation (held out)
Total	**20,000**	

2.2 Features and Labels

Each record has two fields:

- `text`: the raw tweet (string);
- `label`: integer class ID mapped to one of six emotions—*sadness* (0), *joy* (1), *love* (2), *anger* (3), *fear* (4), *surprise* (5).

2.3 Exploratory Analysis

Basic statistics on the training set (Table 2) show no missing values and only 31 duplicate texts out of 16,000 samples. The average tweet length is approximately 19 words (96 characters), with a maximum of 66 words.

Table 2: Training set descriptive statistics

Statistic	Label	Char. length	Word count
Mean	1.57	96.85	19.17
Std. dev.	1.50	55.90	10.99
Min	0	7	1
Max	5	300	66

The class distribution is **imbalanced**: *joy* and *sadness* dominate, while *surprise* is the smallest class (Table 3). Figure 1 illustrates the class distribution and word-count histogram.

Table 3: Training set class distribution

Emotion	Count	Percentage
Joy	5,362	33.5%
Sadness	4,666	29.2%
Anger	2,159	13.5%
Fear	1,937	12.1%
Love	1,304	8.2%
Surprise	572	3.6%
Total	**16,000**	**100%**

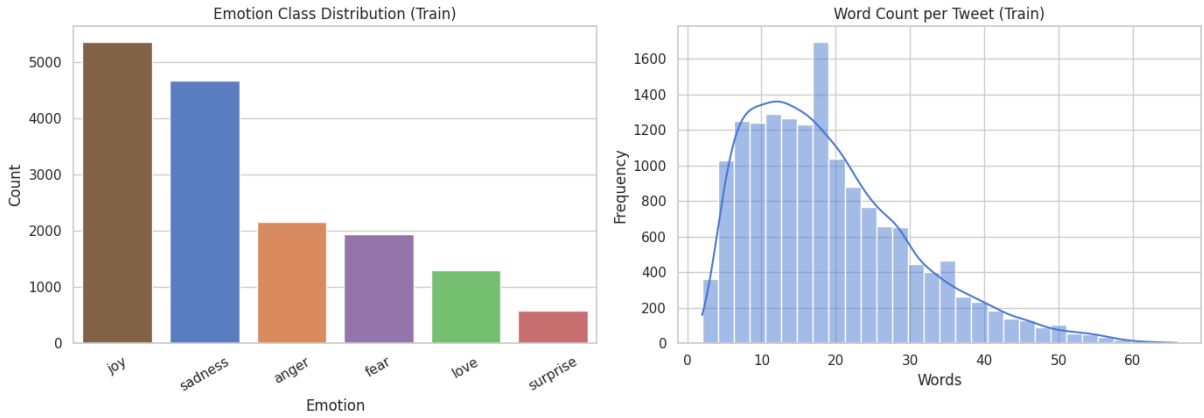

Figure 1: Exploratory data analysis on the training set: emotion class distribution (left) and word-count distribution (right).

3 Methodology

Our approach follows a standard supervised text-classification pipeline. Each design choice below is motivated by properties of short, noisy social-media text.

3.1 Text Preprocessing

For each tweet we applied the following steps:

1. **Lowercasing** — normalizes tokens so that "Happy" and "happy" are treated as the same feature.

2. **Remove URLs and mentions** — URLs and `@user` tags carry little emotional signal and add sparse noise; removing them helps the model focus on lexical content.

3. **Keep hashtag words, remove the # symbol** — hashtags often encode topic or sentiment (e.g., `#excited` → "excited"), so the word itself may still be informative.

4. **Remove non-alphabetic characters** — punctuation and digits are inconsistent across tweets and rarely help distinguish emotions in this dataset.

5. **Stopword removal** — filters high-frequency function words (e.g., "the", "is") that contribute little to class separation in bag-of-words models.

6. **Lemmatization** — reduces inflected forms ("feeling", "felt") to a common base, shrinking the vocabulary and improving generalization.

Limitation. One risk of standard stopword removal is that negation words such as "not", "no", and "never" may be removed depending on the stopword list or preprocessing configuration. Therefore, future versions should explicitly preserve negation terms during preprocessing, or use context-aware models that encode polarity.

3.2 Feature Engineering

Cleaned text was vectorized using **TF-IDF** (Term Frequency–Inverse Document Frequency). TF-IDF down-weights common words while emphasizing terms that are frequent in a tweet but rare across the corpus—a standard and efficient representation for large text datasets. We used:

- `max_features` = 10,000 — limits memory use while retaining the most informative terms;
- `ngram_range` = (1, 2) — captures both single words and short phrases (e.g., "feel sad");
- `min_df` = 2 — ignores very rare terms that may not generalize;
- `sublinear_tf` = True — dampens the effect of very frequent words within a document.

This produced a sparse matrix of shape $(16{,}000 \times 10{,}000)$ for the training set.

3.3 Model Selection and Hyperparameter Tuning

We compared several baselines on the same preprocessed data (Table 4). A **majority-class baseline** (always predicting *joy*, the largest class) achieves only 34.75% accuracy, establishing a lower bound. Among learned models, **Logistic Regression** offered the best balance of performance, training speed, and interpretability for high-dimensional sparse TF-IDF features.

Hyperparameters for Logistic Regression were tuned with `GridSearchCV` using 3-fold **stratified** cross-validation on the training set, optimizing weighted F1-score. The search space was:

- regularization strength $C \in \{0.1,\ 1.0,\ 5.0\}$;
- `class_weight` $\in \{None,\ balanced\}$ — `balanced` adjusts for class imbalance by penalizing errors on minority classes more heavily.

Best configuration: $C = 5.0$, `class_weight = balanced`, with cross-validated weighted F1 = **0.9009**.

4 Implementation

The complete solution is implemented in the Jupyter notebook `emotion_detection.ipynb` (executed version: `emotion_detection (result).ipynb`). In the notebook, Section 7 compares baseline models (majority class, Naive Bayes, Linear SVM), while Section 8 tunes and selects Logistic Regression as the final classifier.

4.1 Project Structure

```
doan/
|-- emotion_detection.ipynb       # Main notebook
|-- emotion_detection (result).ipynb
|-- requirements.txt              # Python dependencies
|-- README.md
|-- outputs/                      # Saved figures and reports
|-- report.tex                    # This report
```

4.2 Key Libraries

- **Hugging Face Datasets** — load the Emotion dataset;
- **NLTK** — stopword removal and lemmatization;
- **scikit-learn** — TF-IDF, Logistic Regression, GridSearchCV, metrics;
- **matplotlib / seaborn** — EDA and result visualizations.

4.3 Repository Link

Code repository: `https://github.com/YOUR_USERNAME/emotion-detection-tweets`

5 Results and Evaluation

5.1 Validation Set

On the validation set (2,000 samples), the tuned model achieved:

- Accuracy: 90.70%
- Precision (weighted): 0.9098
- Recall (weighted): 0.9070
- F1-score (weighted): 0.9078

5.2 Test Set — Model Comparison

To contextualize the final result, Table 4 compares our tuned Logistic Regression model against simpler baselines on the same test set and preprocessing pipeline.

Table 4: Model comparison on the test set

Model	Accuracy	Weighted F1
Majority Class (always predict *joy*)	34.75%	0.1792
TF-IDF + Multinomial Naive Bayes	77.00%	0.7345
TF-IDF + Linear SVM	90.05%	0.9019
TF-IDF + Logistic Regression (tuned)	**90.25%**	**0.9043**

The tuned Logistic Regression model improves accuracy by **55.5 percentage points** over the majority-class baseline and slightly outperforms Linear SVM and Naive Bayes, confirming that the result is well above a trivial solution.

5.3 Test Set — Overall Metrics

Table 5 reports the detailed metrics for the selected Logistic Regression model on the held-out test set.

Table 5: Overall test-set metrics (weighted average)

Metric	Score
Accuracy	90.25%
Precision	0.9091
Recall	0.9025
F1-score	0.9043

5.4 Per-Class Performance

Table 6 shows precision, recall, and F1-score for each emotion class. Classes with more training examples (*sadness*, *joy*) achieved F1-scores above 0.92. *Love* had lower precision (0.7268) due to confusion with *joy*, while *surprise*—the smallest class—had lower precision (0.6905) but relatively high recall (0.8788).

Table 6: Per-class classification report on the test set

Emotion	Precision	Recall	F1-score	Support
Sadness	0.9520	0.9225	0.9371	581
Joy	0.9514	0.9022	0.9261	695
Love	0.7268	0.8868	0.7989	159
Anger	0.8694	0.9200	0.8940	275
Fear	0.9091	0.8482	0.8776	224
Surprise	0.6905	0.8788	0.7733	66
Macro avg	0.8499	0.8931	0.8678	2,000
Weighted avg	0.9091	0.9025	0.9043	2,000

5.5 Confusion Matrix and Visualizations

Figure 2 shows the confusion matrix on the test set. The largest off-diagonal errors involve emotionally adjacent classes. Specifically, **46** *joy* tweets were misclassified as *love*, and **15** *love* tweets were misclassified as *joy*—the most frequent confusion pair overall. Other notable errors include **22** *sadness* $\rightarrow$ *anger*, **14** *fear* $\rightarrow$ *surprise*, and **12** *anger* $\rightarrow$ *sadness*. These patterns align with overlapping vocabulary (e.g., affectionate language shared by *joy* and *love*) and the difficulty of distinguishing intensity or valence in very short texts. Figure 3 summarizes per-class F1-scores.

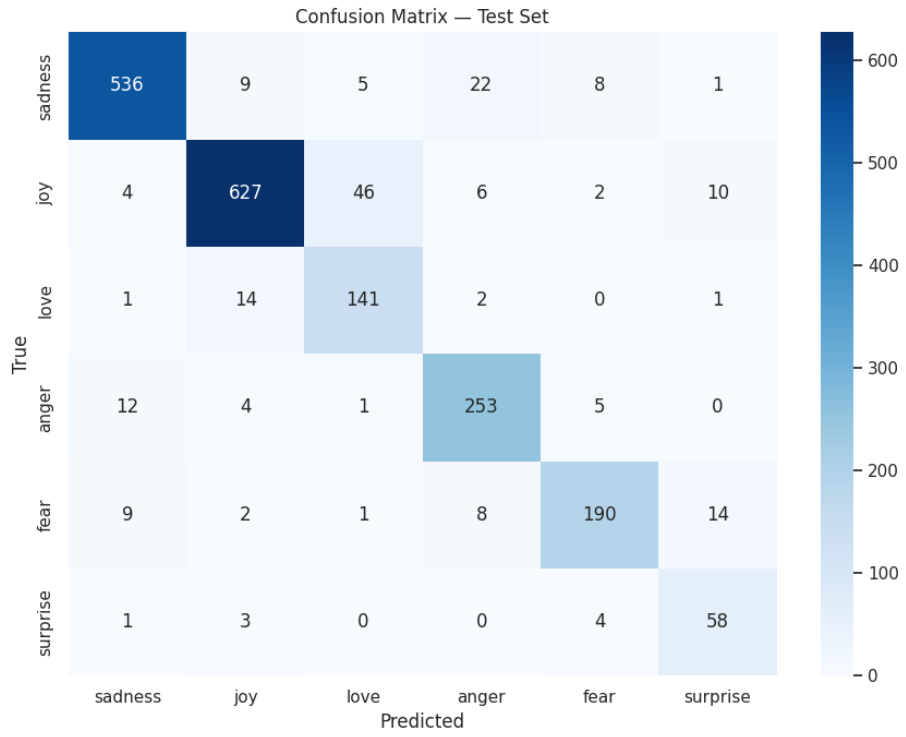

Figure 2: Confusion matrix on the test set (2,000 tweets).

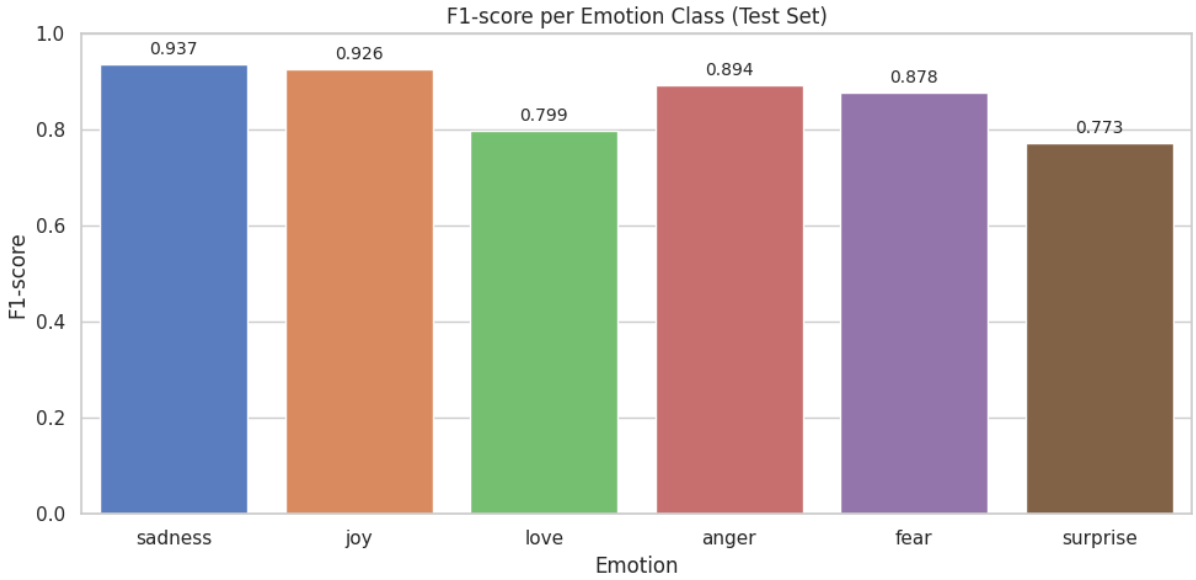

Figure 3: F1-score per emotion class on the test set.

5.6 Error Analysis

Table 7 presents representative misclassified examples drawn from the test set. Most errors stem from semantic overlap between classes, mixed emotions within a single tweet, or ambiguous wording that human annotators and the model may interpret differently.

Table 7: Representative misclassification examples

Label	**Predicted**	**Example (abridged)**	**Possible Reason**
Joy	Love	"I feel like I am in paradise kissing those sweet lips…"	Affectionate vocabulary overlaps with *love*.
Love	Joy	"I'm feeling generous today here's one more…"	Positive, lighthearted tone resembles *joy*.
Sadness	Love	"I'm not sure the feeling of loss will ever go away but it may dull to a sweet feeling of nostalgia…"	"Sweet" and nostalgic wording blurs sadness and love.
Fear	Joy	"I was feeling pretty anxious all day but my first day at work was a very good day…"	Mixed emotions: anxiety followed by relief/positivity.
Surprise	Fear	"I was so uncomfortable and feeling weird feelings but wasn't sure if they were contractions…"	Uncertainty and discomfort resemble *fear*.
Anger	Fear	"I feel my heart is tortured by what I have done"	Intense negative emotion; vocabulary is ambiguous between anger and fear.

These cases illustrate that fine-grained emotion boundaries are inherently fuzzy in short, context-poor social-media text—a key motivation for future context-aware models.

6 Individual Contribution

Table 8: Individual contribution breakdown

Member	Responsibilities
Lâm Nhật Huy	Data collection & loading; data cleaning & preprocessing; EDA with charts; featu
Nguyễn Thị Lam Giang	Model selection & building; training & hyperparameter tuning; evaluation metrics
Total	Both members: integration, testing, and peer review

7 Conclusion and Future Work

7.1 Summary

We successfully built an end-to-end emotion detection pipeline for tweets. After preprocessing and TF-IDF vectorization, a tuned Logistic Regression classifier achieved **90.25% accuracy** and a **weighted F1-score of 0.9043** on the test set, meeting the project requirements for multi-class classification with full metric reporting.

7.2 Key Findings

- The tuned model (**90.25%** accuracy) far exceeds a majority-class baseline (**34.75%**), demonstrating meaningful learning rather than class-frequency effects alone.
- TF-IDF with unigrams and bigrams captures sufficient signal for six-way emotion classification on this dataset, though it cannot fully resolve context-dependent cases.
- Class imbalance affects minority classes; `class_weight=balanced` and $C = 5.0$ improved overall F1 during cross-validation.
- The dominant confusion is *joy* $\leftrightarrow$ *love* (46 + 15 errors), reflecting semantically adjacent positive emotions.
- Standard stopword removal is a practical simplification but may miss nuanced polarity; preserving negation terms is an important preprocessing consideration.

7.3 Future Work

Each proposed improvement directly addresses a limitation observed in this project:

1. **Fine-tune a transformer (e.g. DistilBERT).** Because TF-IDF treats words independently and cannot model context or negation reliably, a pretrained language model could better capture semantics (e.g., distinguishing "not happy" from "happy").

2. **Oversampling or data augmentation for minority classes.** Since *surprise* accounts for only 3.6% of training data, targeted augmentation could reduce the precision/recall gap seen for rare classes.

3. **Deploy as a REST API.** The current solution is notebook-based and offline; packaging the trained pipeline as an API would be the next step toward practical integration into monitoring tools.

4. **Evaluate on newer Twitter/X data.** The benchmark dataset may not reflect current language use; testing on freshly collected posts would reveal whether the model generalizes beyond the benchmark.

References

[1] Bird, S., Klein, E., & Loper, E. (2009). *Natural language processing with Python: Analyzing text with the Natural Language Toolkit*. O'Reilly Media.

[2] dair-ai. (2020). *Emotion dataset* [Data set]. Hugging Face. `https://huggingface.co/datasets/dair-ai/emotion`

[3] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, *12*, 2825–2830.

[4] Saravia, E., Liu, H.-C., Huang, Y.-H., Wu, J., & Chen, Y.-S. (2018). CARER: Contextualized affect representations for emotion recognition. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing* (pp. 3687–3697). Association for Computational Linguistics. `https://doi.org/10.18653/v1/D18-1404`